Introdução à Programação Orientada a Objetos em Java
Parte 3

I. Interface
II. Polimorfismo
  i. Sobreposição de métodos
  ii. Sobrecarga
III. Referências

I. Interface

Em diversos contextos, padronizar pode trazer grandes benefícios. Inclusive, no desenvolvimento de aplicações. Veja a seguir como a ideia de padronização pode ser contextualizada nos conceitos de orientação a objetos.

Em um sistema orientado a objetos, os objetos interagem entre si através de chamadas de métodos (troca de mensagens). Podemos dizer que os objetos se "encaixam" através dos métodos públicos assim como um plugue se encaixa em uma tomada através dos pinos.

Para os objetos de uma aplicação "conversarem" entre si mais facilmente é importante padronizar o conjunto de métodos oferecidos por eles. Assim como os plugues encaixam nas tomadas mais facilmente graças aos padrões definidos pelo governo.

Um padrão é definido através de especificações ou contratos. Nas aplicações orientadas a objetos, podemos criar um "contrato" para definir um determinado conjunto de atributos e métodos que devem ser implementados pelas classes que "assinarem" este contrato. Em orientação a objetos, um contrato é chamado de interface. Uma interface é composta basicamente por métodos abstratos (sem corpo).

Uma interface é definida usando a palavra reservada ***interface*** e todos os métodos não possuem corpo. Ela precisa ser implementada por uma classe (implementar significa codificar corpo nos métodos sem corpo). Os membros da interface funcionam como um contrato mínimo a ser seguido pelas classes que implementam a interface. No exemplo a seguir, a classe Cliente, obrigatoriamente, precisa ter os atributos nome e nasc (data de nascimento) e o método print com as assinaturas (ordem e tipos dos parâmetros) definidas na interface Pessoa. Além disso, ela poderá utilizar o método padrão (default) idade presente na interface. Nesse caso, o método idade justifica-se, pois o cálculo da idade de uma pessoa, sempre será o mesmo.

```
import java.util.Calendar;
import java.util.GregorianCalendar;

public interface Pessoa {

    public String nome = "";
    GregorianCalendar nasc = new GregorianCalendar();

    public void print();

    default int idade(GregorianCalendar nasc) {
        GregorianCalendar hoje = new GregorianCalendar();
        int anohoje = hoje.get(Calendar.YEAR);
        int anoNascimento = nasc.get(Calendar.YEAR);
```

```
        return anohoje-anoNascimento;
    }
}

import java.text.DecimalFormat;
import java.text.SimpleDateFormat;
import java.util.GregorianCalendar;

public class Cliente implements Pessoa {

    private String nome;
    private GregorianCalendar nasc;
    private double saldo;

    public Cliente(String nome, GregorianCalendar nasc, double saldo) {
        this.nome = nome;
        this.nasc = nasc;
        this.saldo = saldo;
    }

    @Override
    public void print() {
        System.out.println("Nome: " + nome);
        System.out.println("Data Nascimento: " + (new
                    SimpleDateFormat("dd/MM/yyyy").format(nasc.getTime())));
        System.out.println("Saldo: "+(new
                    DecimalFormat("#,###,###.00").format(saldo)));
    }
}

import java.util.GregorianCalendar;

public class TesteCliente {

    public static void main(String[] args) {
        GregorianCalendar nasc = new GregorianCalendar();
        nasc.set(2006, 1, 10);
        Cliente cli = new Cliente("Ana", nasc , 950);
        System.out.println(cli.idade(nasc)); //correto: o tipo
                                    Cliente herda o método idade
        cli.print(); //correto: o tipo Cliente possui o método print
    }
}
```

Um objeto que implementa uma interface precisa ter exatamente aquilo que nela foi definido.

II. Polimorfismo

A capacidade de tratar objetos criados a partir das classes específicas como objetos de uma classe

genérica é chamada de **polimorfismo**.

É um dos conceitos mais usados do paradigma da orientação a objetos, pois possibilita a reutilização contínua dos códigos, ou seja, permite algo assumir várias formas. No contexto da programação orientada a objetos, ele nos mostra como um método pode assumir formas diferentes das quais foram implementadas inicialmente e agir de modo que possa ser utilizado por outra classe.

Tanto o polimorfismo quanto a herança são referências no ramo de reutilização de código, pois trabalham em conjunto.

Os tipos de polimorfismo mais utilizados são: sobreposição e sobrecarga.

Sobreposição de métodos (Override)

Na sobreposição de métodos ocorre a substituição (Override) de um método da superclasse por outro na classe herdeira (filha ou subclasse).

No contexto do polimorfismo, podemos reescrever métodos nas classes filhas que foram originalmente criados na classe pai. Esses métodos sobrepostos devem ter o mesmo nome, tipo de retorno e quantidade de parâmetros do método original, mas podem ser implementados com especificações específicas da classe atual, podendo adicionar funcionalidades extras ou não.

Veja o exemplo do método print() na superclasse Veiculo que é sobreposto pelo método print da subclasse Carro:

```
public class Veiculo {
    String tipo;
    String combustivel;

    public Veiculo(String tipo, String combustivel) {
        this.tipo = tipo;
        this.combustivel = combustivel;
    }
    public void print(){
        System.out.println(tipo+" "+combustivel);
    }
}
public class Carro extends Veiculo {
    String marca;
    String modelo;
    public Carro(String tipo, String combustivel, String marca, String modelo) {
        super(tipo, combustivel);
        this.marca = marca;
        this.modelo = modelo;
    }
    @Override
    public void print(){
        System.out.printf("%s %s %s %s",marca,modelo,combustivel,tipo);
    }
}
```

Perceba que ao instanciar um objeto da classe Carro, e ao chamar o método print, o que será mostrado na saída padrão, é a informação existente no método print da classe Carro e não da sua superclasse Veiculo.

```
public class TesteCarro {
```

```
    public static void main(String[] args) {
        Carro c = new Carro("terrestre","flex","Ford","Ka");
        c.print();
    }
}
```

Saida padrão:

terrestre flex Ford Ka

Sobrecarga de métodos (Overload)

A sobrecarga ocorre quando são usadas assinaturas (tipos e ordem dos parâmetros) diferentes em métodos com o mesmo nome.

A sobrecarga de métodos, também conhecida como overload, envolve criar variações de um mesmo método em uma classe. Isso significa que podemos ter dois ou mais métodos com nomes idênticos, desde que suas listas de argumentos sejam diferentes, permitindo a separação entre eles.

Os métodos Construtores são um exemplo clássico de sobrecarga. Veja no exemplo a seguir que a classe Circulo possui dois construtores com assinaturas diferentes. O padrão, que não recebe nenhum parâmetro de inicialização do atributo raio e outro no qual recebe esse valor. Assim, podemos instanciar um objeto dessa classe de duas maneiras diferentes:

```
public class Circulo {

    private double raio;

    // Construtor padrão
    public Circulo() {
        this.raio = 1.0;
    }

    // Construtor com raio especificado
    public Circulo(double raio) {
        this.raio = raio;
    }

    // Método para calcular a área do círculo
    public double calcularArea() {
        return Math.PI * raio * raio;
    }

    // Método para calcular o perímetro do círculo
    public double calcularPerimetro() {
        return 2 * Math.PI * raio;
    }
}
```

Veja na classe executora TesteCirculo, como ficaram os instanciamentos:

```
public class TesteCirculo {

    public static void main(String[] args) {
```

```
        Circulo circulo1 = new Circulo(); // Círculo com raio padrão
        Circulo circulo2 = new Circulo(3.5); // Círculo com raio 3.5

        System.out.println("Área do círculo 1: " + circulo1.calcularArea());
        System.out.println("Perímetro do círculo 1: " +
                                circulo1.calcularPerimetro());

        System.out.println("Área do círculo 2: " + circulo2.calcularArea());
        System.out.println("Perímetro do círculo 2: " +
                                circulo2.calcularPerimetro());
    }
}
```

Resultado na saída padrão:

Área do círculo 1: 3.141592653589793
Perímetro do círculo 1: 6.283185307179586
Área do círculo 2: 38.48451000647496
Perímetro do círculo 2: 21.991148575128552

Vejamos agora, outro exemplo dos métodos somar da classe Calculadora sobrecarregados. Perceba que a sobrecarga ocorre nos tipos e quantidades de parâmetros que são passados em cada método, que possuem o mesmo nome:

```
public class Calculadora {

    // Método para somar dois inteiros
    public int somar(int a, int b) {
        return a + b;
    }

    // Método sobrecarregado para somar três inteiros
    public int somar(int a, int b, int c) {
        return a + b + c;
    }

    // Método sobrecarregado para somar dois números de ponto flutuante
    public double somar(double a, double b) {
        return a + b;
    }
}
```

Agora a classe TesteCalc demonstra os instanciamentos da classe Calculadora:

```
public class TesteCalc {

    public static void main(String[] args) {

        Calculadora calc = new Calculadora();

        // Exemplo de uso dos métodos sobrecarregados
        System.out.println("Soma de 5 e 3: " + calc.somar(5, 3));
        System.out.println("Soma de 2, 4 e 6: " + calc.somar(2, 4, 6));
```

```
        System.out.println("Soma de 2.5 e 3.7: " + calc.somar(2.5, 3.7));
    }
}
```

Resultado na saída padrão:

```
Soma de 5 e 3: 8
Soma de 2, 4 e 6: 12
Soma de 2.5 e 3.7: 6.2
```

III. Referências

Partes desta apostila, foram baseadas no material didático do prof. Dr. Arley Ferreira de Souza coordenador do curso de DSM da Fatec de Jacareí.

Java: How to program, 4th. Ed., Harvey M. Deitel e Paul J. Deitel, Prentice-Hall, 2001, ISBN 0-13-034151-7.

Design patterns: Elements of reusable object-oriented software, Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, Addison-Wesley, 1995, ISBN 0-201-63361-2.

Object-oriented design in Java, Stephen Gilbert, Bill McCarty, The Waite Group Press, 1998, ISBN 1-57169-134-0.

The Unified Modeling Language User Guide, Grady Booch, James Rumbaugh, Ivar Jacobson, Addison-Wesley, 1999, ISBN 0-201-57168-4.

The Java Tutorial: A practical guide for programmers, Mary Campione e Kathy Walrath, acesso em 30/04/2024.

Java (linguagem de programação), Wikipédia, acesso em 30/04/2024.

DevMedia, Sobrecarga e sobreposição de métodos em orientação a objetos, em https://www.devmedia.com.br/sobrecarga-e-sobreposicao-de-metodos-em-orientacao-a-objetos/33066#:~:text=A%20Sobreposi%C3%A7%C3%A3o%20de%20m%C3%A9todos%20(override,nome%2C%20tipo%20de%20retorno%20e, acesso em 11/06/2024.